A High Throughput Kyber NTT

Jonas Bertels
COSIC
KU Leuven
Leuven, Belgium
jonas.bertels@esat.kuleuven.be

Ingrid Verbauwhede
COSIC
KU Leuven
Leuven, Belgium
ingrid.verbauwhede@kuleuven.be

***Abstract*—NIST recently selected Kyber as a standard for key encapsulation and decapsulation. As such, servers will soon need dedicated hardware for these encapsulation protocols. The computationally critical operation of Kyber is its Number Theoretic Transform, which is commonly accelerated by dedicated hardware such as FPGAs. This work presents an extremely high-throughput design for the Kyber NTT. By utilizing the LUT-based modular multiplication technique used by Bertels et al. [BPV25], its area delay product is generally between one and two orders of magnitude better than similar designs in literature. For instance, where Yaman et al. [YMÖS21] with 16 Processing Elements requires 9500 LUTs and 16 DSPs to perform an NTT in 69 clock cycles, our design requires 67210 LUTs and no DSPs to perform an NTT every clock cycle.**

***Index Terms*—CRYSTALS-Kyber, Hardware Design, FPGA**

I. Introduction

Quantum Computing doesn't just offer great opportunities for future calculation, but also poses threats to public key cryptographic algorithms. Attacks with algorithms such as Shor's factorization algorithm can rapidly solve large integer factorization or the discrete logarithm problem [Sho97], thereby breaking RSA and elliptic-curve cryptography, respectively. To deal with this new threat, cryptographic algorithms resistant to these attacks have been developed. We refer to these algorithms collectively as Post-Quantum Cryptography (PQC). The US National Institute of Standards and Technology (NIST) launched a standardization effort for this family of algorithms in 2016 [Div16]. NIST announced the first schemes selected for standardization in 2022.

As Key-Encapsulation Mechanism, a scheme named Kyber [Div23b] was picked. Kyber ('ML-KEM' [Div23a]), relies on repeated modular polynomial multiplication. By employing the Number Theoretic Transform (NTT) on large-degree polynomials of size N (e.g., $N = 256$ for Kyber), each polynomial multiplication can be performed in $\frac{3}{2}N \times \log N$ modular multiplications. As the NTT often takes up over half of the execution time of Kyber, it is a worthwhile target for hardware acceleration[1].

As Kyber becomes standardized, it will increasingly be used on servers and in server-side applications, where throughput and low latency are key requirements. On these servers, FPGAs are often readily available to help accelerate key parts of the computation.

[1]https://github.com/mupq/pqm4

Related Work Since the NTT qualifies as a key part of the Kyber Key Encapsulation Mechanism, there is no shortage of hardware designs which set out to improve the throughput and latency of the NTT for Kyber parameters using FPGAs [NKLO23], [NPH23], [YMÖS21], [LQYW24]. Many of these designs do not specifically target servers or larger FPGAs, but offer solutions which would fit both on embedded devices and as additional hardware on servers. The downside of this approach is that this leaves significant resources on server-side FPGAs unused, as modern FPGAs such as the AMD Alveo U55c have significantly more LUTs, FFs and memory elements than their embedded equivalents such as the AMD Artix series. One solution is to place multiple instances of these smaller general-purpose designs on larger FPGAs. The other approach, taken in this paper, is to create a server-specific NTT FPGA design which has an area delay product between an order and two orders of magnitude better than general-purpose designs.

Contribution. The main contributions of this paper are thus the following:

- A DSP-free 30 LUT modular multiplication unit based on [BPV25]
- A 69-LUT butterfly unit employing lazy reduction, an innovative LUT-based reduction unit and the aforementioned modular multiplication unit
- A radix-256, 7-stage Number Theoretic Transform taking up 67210 LUTs on the AMD Alveo U55c with a throughput of one Number Theoretic Transform per clock cycle

Outline. Firstly, we introduce butterfly circuits, the Number Theoretic Transform and our modular multiplication technique in Section II. Secondly, we show how we implemented modular multiplication and the butterfly circuit on the FPGA in Section III. Finally, we give results and a comparison with other designs in Section IV.

II. Preliminaries

This section introduces the butterfly operation and NTTs.

A. The Cooley-Tukey and Gentleman-Sande Butterfly

A Number Theoretic Transform can be seen as a Fourier transform performed over the integers modulo $q = 3329$. Usually, the NTT consists of performing $\frac{N}{2} \times \log N$ butterflies, although in the case for Kyber parameters where $\log N = 8$, it is not possible to do a full Number Theoretic Transform and

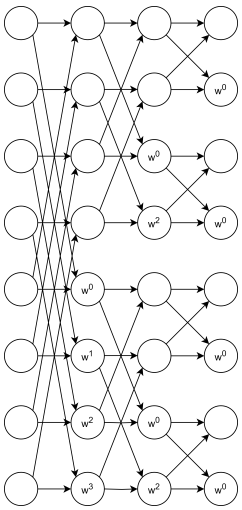

Fig. 1. A series of butterfly units connected to each other. The twiddle factors (ζ) used in each butterfly unit can be found from the reference implementation or official Kyber specifications [Div23b]

only 7 stages are performed, leading to $\frac{256}{2} \times 7 = 896$ butterflies. Butterflies and their implementations designed for Kyber-parameter (Inverse) Number Theoretic Transforms ((I)NTTs) come in two forms and contain a modular addition, a modular subtraction, and a single modular multiplication with a twiddle factor. The forward NTT, whose butterfly is usually chosen to be a Cooley-Tukey butterfly, is of the form:

$$a_{out} = (a_{in} + b_{in} \cdot w) \bmod q$$
$$b_{out} = (a_{in} - b_{in} \cdot w) \bmod q$$

For the inverse Number Theoretic Transform, the Gentleman-Sande butterfly is commonly chosen, which is of the following form:

$$a_{out} = (a_{in} + b_{in}) \cdot w \bmod q$$
$$b_{out} = (a_{in} - b_{in}) \cdot w \bmod q$$

with a_{in} and b_{in}, 12 bit inputs, w a 12 bit constant multiplicand, $w = \omega^i, i \in [0 \dots 127]$ with $\omega^{n/2} = -1 \bmod q$. For the sake of simplicity, in this paper only the forward NTT is considered, although all techniques presented here can be applied nearly unmodified to the inverse NTT. As such, the butterfly used from this point onwards will always be the Cooley-Tukey butterfly.

A Number Theoretic Transform is then these butterfly circuits, with their respective twiddle factors, connected to each other in the manner given by Figure 1.

B. Modular Multiplication

Most modular multiplication designs for Kyber parameters use some variant on Barrett, Montgomery, K-reduction or LUT-reduction [BNV24]. All of these designs work from the assumption that both multiplier inputs are variable. The most efficient designs assume the modulus remains constant and therefore these designs hardwire large parts of the modular reduction and reduce LUT usage. In Bertels et al.'s paper [BPV25] on a hardware accelerator for the TFHE algorithm, a new modular multiplier for FPGAs was presented, which borrows ideas from in-memory computation and LUT-based reduction. It works for any modular multiplier where the multiplication unit is fixed and takes advantage of the linearity of both the multiplication and the modular reduction operation. Consider the following function:

$$f(a) = a \times b \bmod q$$

where b and q are 12-bit constants.
We see that f is a linear function by looking at the evaluation of an input $C_0 \times a_0 + C_1 \times a_1$:

$$f(C_0 \times a_0 + C_1 \times a_1)$$
$$= (a_0 \times C_0 \times b \bmod q + a_1 \times C_1 \times b \bmod q) \bmod q$$

Moreover any $\log_2(q) = 12$-bit integer a can be split up in 3 sections of 4 bits.

$$a = a_{[11:8]} \times 2^8 + a_{[7:4]} \times 2^4 + a_{[3:0]} \times 2^0 \tag{1}$$

Thus our modular multiplication reduces to $\log_2(q)/4 = 3$ functions with 4-bit inputs.

$$f(a) = (f(a_{[11:8]} \times 2^8) + f(a_{[7:4]} \times 2^4) + \dots) \bmod q$$

When we consider each term individually, we see that these functions take 4 bit inputs and return 12 bit outputs.

$$f(a_{[11:8]} \times 2^8) = g_2(a_{[11:8]}) = a_{[11:8]} \times b \times 2^8 \bmod q$$
$$f(a_{[7:4]} \times 2^4) = g_1(a_{[7:4]}) = a_{[7:4]} \times b \times 2^4 \bmod q$$
$$f(a_{[3:0]} \times 2^0) = g_0(a_{[3:0]}) = a_{[3:0]} \times b \times 2^0 \bmod q$$

From 6 Look up Tables, each taking 4 bit inputs and outputting 2 bits, we can create the required g_i function. From three of these functions, we can create the three terms of Equation (1). Note that we can do even better, as the primitive available on most 7-series and Ultrascale+ FPGAs is a 5-to-2 Look-Up Table, so we can take 15 bits as input and output three 12 bit terms, thereby reducing the original input as we do the multiplication.

We note that one important drawback of this design is that it forces us to use constant multipliers. As shown by Bertels et al. [BPV25], this means we must either place the entire NTT on chip, or to use the constant-multiplier NTT employed in that paper. However, since the Kyber parameter set is smaller than that commonly employed by TFHE, placing the entire NTT on chip is feasible as long as we take great care to optimize the LUT usage of each butterfly.

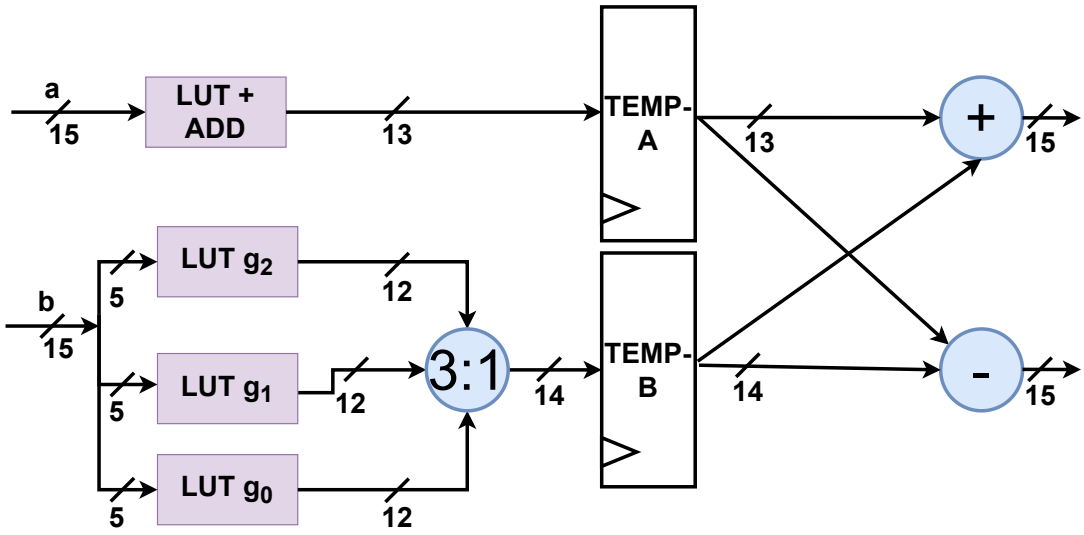

Fig. 2. Our lazy reduction butterfly circuit, consisting of 18 LUTs for the modular multiplication of **b**, a 12 LUT 3:1 adder to sum the multiplication results, 12 LUTs to reduce the **a** input and 27 LUTs to finish the butterfly. This means a total of 69 LUTs are required for each butterfly circuit.

III. Hardware Implementation

The design of a single butterfly is given by Figure 2. To reduce the LUT usage per butterfly to an absolute minimum, we use two techniques from Florent de Dinechin and Martin Kumm's excellent book Application-Specific Arithmetic [dDK24].

The first technique is taken wholesale from the book and is their 3:1 adder for the price of a regular adder. While this adder has a sub-optimal carry chain from a timing perspective, on Ultrascale+ devices it still allows for frequencies over 400 MHz, even when combined with the initial LUT stage seen at the bottom left in Figure 2.

The second technique is developed from their observation that within a LUT6+Carry block, the hardware designer can combine a 5:1 bit function with the addition of a single bit. This allows us to create a very efficient LUT-reduction circuit which takes the upper 3 bits as input for each LUT, as well as a single bit from each of the lower 12 bits. In other words, we have

$$\text{TEMP-A} = g_3(a[14:12]) + a \tag{2}$$

where g_3 is the function:

$$g_3(x) = x \times 2^{12} \bmod q \tag{3}$$

TEMP-A can then be computed using only 12 LUTs as g_3 and the addition can be merged within a single LUT block.

In total, 7×128 butterfly units are placed on the chip, as we place the entire Number Theoretic Transform on the AMD Alveo U55c FPGA. At the output, all 256 outputted coefficients go through a smaller reduction stage analogous to the LUT+ADD stage in the butterfly, followed by a multiplexer for the final reduction. The total area usage is then 67210 LUTs. The design is able to handle a full NTT every cycle, and utilizes two pipeline stages per butterfly (one visible in the center of figure 2, one at the end). Moreover, two pipeline stages are used for the final reduction. This results in a total latency of $7 \times 2 + 2 = 16$ cycles and a throughput of one NTT per cycle.

IV. Results

In this section, we compare our modular multiplication, butterfly unit and finally our entire Number Theoretic Transform with other designs.

Our modular multiplication, by taking full advantage of the fact that modular reduction is a reduction, requires fewer LUTs than an ordinary multiplication. This is a reversal of the situation in most designs, which contain a separate multiplication and reduction module. The modular multiplication unit is synthesized separately in Table III for a better comparison between the different designs.

Our butterfly unit is significantly more efficient than the average butterfly unit. The reasons for this are twofold: Firstly, we use lazy reduction throughout the whole circuit and let the intermediate values expand to 15 bits. Thus, we only have two reduction parts of the circuit, which are our very efficient modular multiplier and our LUT+ADD circuit. Secondly, there are no multiplexers, BRAMs or control logic inside or between the butterflies. This guarantees only computationally useful logic is performed. We note that for the comparison in Table IV, we utilize the results given by Bertels et al. [BNV24] where Yaman et al.'s butterfly unit is used in conjunction with

[2]We use Sun et al.'s definition [SB24], where for Artix-7 FPGAs, A=Slice+DSP×100+BRAM×200 and for Virtex-7 FPGAs, A=Slice+DSP×100+BRAM×100 and Slices=LUTs/4+FFs/8 where not available. In our case on Ultrascale+, Slices are twice the size, so for our design A=Slice×2

TABLE I
NUMBER THEORETIC TRANSFORM COMPARISON

Paper	Platform	LUT	FF	BRAM	DSP	Slices	A^2	Cycles (Amort.)	Freq.	Latency (μs)
Guo et al. [GLK22]	Artix-7	1740	643	0	4	575	975	225	200	1.125
Ye et al. [YCH22]	Virtex-7	1362	1036	0.5	7	N/A	1220	556	235	2.37
Yaman et al. [YMÖS21]	Artix-7	2543	792	9	4	N/A	2935	232	182	1.27
Yaman et al. [YMÖS21]	Artix-7	9508	2684	35	16	N/A	11313	69	172	0.4
Guo et al. [GL24]	Artix-7	784	441	3	2	259	1059	313	200	1.57
Ni et al. [NKLO23]	Artix-7	1154	1031	0	2	445	645	456	300	1.5
Sun et al. [SB24]	Virtex-7	982	1151	10.5	5	392	1018	306	296	1.03
Ours	**Ultrascale+**	**67210**	**60545**	**0**	**0**	**10717**	**21434**	**1**	400	**0.04**

TABLE II
NUMBER THEORETIC TRANSFORM COMPARISON

Paper	TP (NTT/ms)	TP/LUT	TP/A
Guo et al. [GLK22]	889	0.511	0.912
Ye et al. [YCH22]	423	0.311	0.347
Yaman et al. [YMÖS21]	784	0.308	0.267
Yaman et al. [YMÖS21]	2493	0.262	0.220
Guo et al. [GL24]	639	0.815	0.603
Ni et al. [NKLO23]	658	0.570	1.020
Sun et al. [SB24]	967	0.985	0.950
Ours [BPV25]	**400 000**	**5.951**	**18.662**

TABLE III
MODULAR MULTIPLICATION AREA USAGE

Paper	# LUT	# FF	# DSP
Yaman et al. [YMÖS21]	167	1	1
Xing and Li [XL21]	90	1	1
Nguyen et al. [GLK22]	248	111	0
Li et al. [LQYW24]	38	0	2
Ni et al. [NKLO23]	50	34	1
Bertels et al. [BNV24]	49	32	1
Ours (Fig. 2)	**30**	**14**	**0**

the modular multiplication unit of the various designs for a fair comparison.

TABLE IV
BUTTERFLY AREA USAGE

Paper	Technique	# LUT	# DSP
Yaman et al. [YMÖS21]	Bit-wise	269	1
Xing and Li [XL21]	Barrett	192	1
Nguyen et al. [GLK22]	LUT/K-red	342	0
Li et al. [LQYW24]	$K^{1.5}$-red	140	2
Ni et al. [NKLO23]	LUT4/K-red	185	1
Bertels et al. [BNV24]	LUT6/K-red	151	1
Ours (Fig. 2)	[BPV25]	**69**	**0**

As our Number Theoretic Transform is just a combination of a large number of our butterfly circuits, it gains all the advantages these optimized butterfly circuits have while incurring none of the additional logic and multiplexer costs other circuits impose by routing their butterfly circuits to memory elements to temporarily store data. As long as data can be fed in at a constant rate, a Number Theoretic Transform can complete every clock cycle at 400 MHz without any stall cycles. Moreover, while 67210 LUTs are challenging to place on an embedded device, they represent only a small fraction (5%) of the available area on the Alveo U55c. The comparison between varion NTT designs is given by Table I and Table II.

V. CONCLUSION

In conclusion, this paper presents an extremely high-throughput NTT design for server-grade FPGAs, capable of performing 400 million Kyber NTTs per second. This design takes up only 5% of the area on an Alveo U55c device, leaving ample space for hashing modules, INTT modules, pointwise multiplication modules and other required hardware. By throughput, it outperforms most designs by two or three orders of magnitude. By throughput per area, it outperforms most designs by one or two orders of magnitude.

REFERENCES

[BNV24] Jonas Bertels, Quinten Norga, and Ingrid Verbauwhede. A better kyber butterfly for fpgas. In *2024 34th International Conference on Field-Programmable Logic and Applications (FPL)*, pages 171–177, 2024.

[BPV25] Jonas Bertels, Hilder V. L. Pereira, and Ingrid Verbauwhede. FINAL bootstrap acceleration on FPGA using dsp-free constant-multiplier ntts. *IACR Cryptol. ePrint Arch.*, page 137, 2025.

[dDK24] Florent de Dinechin and Martin Kumm. *Application-Specific Arithmetic*. Springer, 2024.

[Div16] NIST Computer Security Division. Post-quantum cryptography standardization. https://csrc.nist.gov/projects/post-quantum-cryptography, 2016. [Online; accessed 20-December-2023].

[Div23a] NIST Computer Security Division. Comments requested on three draft FIPS for post-quantum cryptography. https://csrc.nist.gov/news/2023/three-draft-fips-for-post-quantum-cryptography, 2023. [Online; accessed 30-October-2023].

[Div23b] NIST Computer Security Division. FIPS 203 (draft): Module-lattice-based key-encapsulation mechanism standard. https://csrc.nist.gov/pubs/fips/203/ipd, 2023. [Online; accessed 30-October-2023].

[GL24] Wenbo Guo and Shuguo Li. Split-radix based compact hardware architecture for crystals-kyber. *IEEE Trans. Computers*, 73(1):97–108, 2024.

[GLK22] Wenbo Guo, Shuguo Li, and Liang Kong. An efficient implementation of KYBER. *IEEE Trans. Circuits Syst. II Express Briefs*, 69(3):1562–1566, 2022.

[LQYW24] Lu Li, Guofeng Qin, Yang Yu, and Weijia Wang. Compact instruction set extensions for kyber. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 43(3):756–760, 2024.

[NKLO23] Ziying Ni, Ayesha Khalid, Weiqiang Liu, and Maire O'Neill. Towards a lightweight CRYSTALS-Kyber in fpgas: an ultra-lightweight BRAM-free NTT core. In *IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, July 2023.

[NPH23] Trong-Hung Nguyen, Cong-Kha Pham, and Trong-Thuc Hoang. A high-efficiency modular multiplication digital signal processing for lattice-based post-quantum cryptography. *Cryptography*, 7(4), 2023.

[SB24] Junyan Sun and Xuefei Bai. A high-speed hardware architecture of an ntt accelerator for crystals-kyber. *Integrated Circuits and Systems*, 1(2):92–102, 2024.

[Sho97] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.

[XL21] Yufei Xing and Shuguo Li. A compact hardware implementation of cca-secure key exchange mechanism CRYSTALS-KYBER on FPGA. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021(2):328–356, 2021.

[YCH22] Zewen Ye, Ray C. C. Cheung, and Kejie Huang. Pipentt: A pipelined number theoretic transform architecture. *IEEE Trans. Circuits Syst. II Express Briefs*, 69(10):4068–4072, 2022.

[YMÖS21] Ferhat Yaman, Ahmet Can Mert, Erdinç Öztürk, and Erkay Savas. A hardware accelerator for polynomial multiplication operation of CRYSTALS-KYBER PQC scheme. In *Design, Automation & Test in Europe Conference & Exhibition, DATE 2021, Grenoble, France, February 1-5, 2021*, pages 1020–1025. IEEE, 2021.